

SPRINT 1

Created: 2026-10-08 23:37:42

Sprint 1 Week 2: Kinematics & Physics Masterclass

Comprehensive Theory, Step-by-Step Actions, and Lessons Learned

Researchers: Ahmed & Shahd | Advisor: Prof. Shan An | Lab: DEX-ROB Lab (Tianjin University)

💡 CONCEPT: THE WHY & INTUITION

Welcome to the Masterclass for **Sprint 1, Week 2**! Today we unlock the fundamental kinematics of the AR5 robot, transition from URDF to OpenUSD, and solve the silent bugs that crash PhysX. By the end of this module, you will understand exactly how the robot moves, how we command it, and why the physics engine behaves the way it does.

1. URDF vs OpenUSD: The Paradigm Shift

💡 CONCEPT: THE WHY & INTUITION

Why abandon URDF?

Think of URDF like a simple stick-figure drawing. It's a tree of links and joints. It's great for basic kinematics but terrible for advanced simulations because it can't handle closed loops (like parallel linkages) or realistic physics (like squishy tomatoes).

OpenUSD is like a full 3D CAD engine combined with a database.

- **URDF:** 1 robot = 1 XML file. 100 robots = 100 XML files.

- **OpenUSD (LIVRPS):** 100 robots = 1 master template, with 100 *non-destructive overrides* (just changing the coordinates of where each robot sits, without copying the 3D meshes).

OpenUSD supports **PhysX 5 FEM** (Finite Element Method), which allows us to simulate the complex, deformable physics of cutting a tomato.

2. Isaac Sim Physics: Articulation Roots & The Instanceable Trap

⚠️ TRAP & SILENT FAILURE

The Instanceable Trap

If you import a robot and click `Instanceable`, Isaac Sim treats the entire robot as a single rigid block to save memory.

The Silent Failure: PhysX looks at the robot and sees **zero joints**. When you try to run your Python controller, you get `TypeError: object of type 'NoneType' has no len()` because PhysX couldn't find the robot's skeleton!

STEP-BY-STEP ACTION: THE HOW

****How to fix the NoneType Error:****

1. Select your robot's top-level Xform (e.g., /World/ar5_robot).
2. Scroll down in the Properties tab and ****Uncheck Instanceable****.
3. Add the `ArticulationRootAPI` to this parent Xform . This tells PhysX: *"Hey, everything inside this folder is connected by joints. Treat it as a single articulated robot."*

3. Joint Drives: Stiffness vs Damping

CONCEPT: THE WHY & INTUITION

****What do 0.5 and 0.05 mean in ADR-006?****

When simulating physical motors, we tune two main properties:

1. ****Stiffness (400 N·m/rad):**** Think of this as a ****Virtual Spring****. If the robot arm is pushed away from its target angle, how aggressively does it snap back? High stiffness = rigid arm.
 2. ****Damping (40 N·m·s/rad):**** Think of this as ****Motor Throttle or Honey****. It opposes the speed of the arm. It prevents the arm from bouncing back and forth (oscillating) like a bungee cord.
- **Velocity Control Rule:**** If you want to control the robot by commanding **speed** (e.g. "spin at 5 rad/s"), you must set ****Stiffness = 0**** and ****Damping > 0****. If stiffness > 0, the robot will try to fight you to stay at a specific angle!

4. The Armature: Virtual Rotor Inertia

CONCEPT: THE WHY & INTUITION

****Why add 0.05 armature to the arm and 0.005 to the hand?****

Real robot motors have heavy metal rotors spinning inside them at high speeds, acting like gyroscopes. This makes the robot stable.

In simulation, if we don't simulate this rotor mass, a tiny calculation error will cause the robot to vibrate violently and explode (numerical instability).

****Armature**** adds "virtual weight" to the joint to stabilize the math, acting just like the real metal rotor.

5. ROS 2: Twist, TF, and QoS Profiles

CONCEPT: THE WHY & INTUITION

****Twist Messages:**** How we tell the robot to move in 3D space. It has two parts:

- ****Linear (x, y, z):**** Moving forward/back, left/right, up/down (like a car).
- ****Angular (x, y, z):**** Pitch, Roll, Yaw (tilting or spinning).

 CONCEPT: THE WHY & INTUITION

****Odometry vs TF (Coordinate Trees):****

Think of Odometry as the robot's **GPS tracker**. TF (Transform Framework) is the **family tree** of where every part of the robot is located relative to the base. `/odom` tells us where the base is in the world, and TF tells us where the knife is relative to the base.

 TRAP & SILENT FAILURE

****Quality of Service (QoS): Best Effort vs Reliable****

When reading high-speed data (like 1 kHz joint packets):

- ****Reliable:**** Like sending an email. If packet #52 drops, the system stops and waits for it. This causes immense lag in real-time robotics!
- ****Best Effort:**** Like watching a live Twitch stream. If packet #52 drops, who cares? Packet #53 is arriving in 1 millisecond. Keep moving!

6. Kinematics and the Manipulator Jacobian

 CONCEPT: THE WHY & INTUITION

****Why do we need the Jacobian?****

In the end, we want to know the **forces** acting on the tomato (Contact Wrench: $\mathbf{F}_{\text{ext}}$).

We can read the electrical currents in the robot's motors, which gives us Joint Torques ($\boldsymbol{\tau}_{\text{ext}}$). But how do we convert 6 spinning motors into a straight-line cutting force?

****The Jacobian Matrix ($\mathbf{J}$)**** is the magic translator. It translates **joint speeds** into **end-effector speeds**.

By applying the math formula $\mathbf{F}_{\text{ext}} = (\mathbf{J}^T)^{\dagger} \boldsymbol{\tau}_{\text{ext}}$, we convert the motor torques directly into the XYZ cutting force on the tomato!

 STEP-BY-STEP ACTION: THE HOW

****Sprint 1, Week 2 Execution:****

We have implemented the exact geometry (D-H parameters) of the AR5 robot in `ar5_kinematics.py`. This script automatically calculates the Forward Kinematics (where the hand is) and the Jacobian (how to translate forces).

This avoids using heavy frameworks like MoveIt for our RL environment, giving us the raw, lightning-fast math needed for training millions of steps in Isaac Sim.